

ML 2425-01 Investigate Image Reconstruction by Using Classifiers

Wahab Abdul
abdul.wahab@stud.fra-uas.de

Rashid Romana
romana.rashid@stud.fra-uas.de

Nabila Nishat Tasnim
lnishat.nabila@stud.fra-uas.de

Abstract— Image reconstruction is one of the most important tasks in computer vision and machine learning, owing to its importance in pattern recognition, autonomous systems, and medical imaging, where it improves analytical accuracy, decision-making, and diagnostic precision. In this project, the feasibility of employing K-Nearest Neighbors (KNN) and HTM classifiers within the Hierarchical Temporal Memory (HTM) framework to reconstruct images from Sparse Distributed Representations (SDRs) is examined. Biologically based HTM is an ideal model for sequence learning and anomaly detection, as it replicates the principles of cortical learning. The purpose of this project is to employ classification methods to reverse the SDR encoding process and retrieve the original input images. We propose a modified classification-based reconstruction framework that utilizes the IClassifier interface to broaden the scope of the SpatialLearning experiment. The proposed method approximates the original inputs by utilizing KNN and HTM classifiers and stores image data in SDRs using an ImageEncoder. The dataset consisted of grayscale images encoded into 64×64 SDRs, representing 4096-bit sparse patterns. In addition to

qualitative visual comparisons, the NeocortexApi quantitative similarity measures are employed to evaluate the reconstruction accuracy. Every classifier's ability to replicate high-dimensional data is demonstrated through an exhaustive performance analysis. According to experimental findings, classification-based SDR inversion is feasible and exhibits a range of reconstruction quality. On average, HTM achieved a hybrid similarity score of 83.5%, outperforming k-NN's 76.2% across the 20 test images, this dataset is small but the framework is scalable so there is scope to use bigger dataset. contingent upon the classifier utilized. This investigation enhances HTM-based image reconstruction and establishes a foundation for subsequent investigations that aim to enhance the reversibility of SDRs within biologically inspired machine learning architectures. A key contribution of this project is the extension of the NeoCortexApi framework with a classifier-driven reconstruction pipeline using the IClassifier interface.

Keywords—Image reconstruction, Classifiers, K-Nearest Neighbors (KNN), Hierarchical Temporal Memory (HTM)

1. INTRODUCTION

Reconstructing or approximating an image from incomplete, noisy, or partial data is the basic challenge in image reconstruction in computer vision, machine learning, and artificial intelligence. Improving the performance of many applications, including medical imaging, autonomous systems, and pattern recognition [1, 2, 3], depends on this process. Widely-used conventional image reconstruction techniques include compressive sensing, imprinting, and interpolation. But the growth of machine learning techniques, especially classification-based methods like K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM) [3], has opened up new ways to improve the accuracy and speed of picture reconstruction. These methods help models understand the fundamental patterns in an image collection, enabling more exact reconstructions even in extremely deteriorated circumstances [4]. Particularly successful in tasks involving spatial and temporal learning, HTM is a scientifically inspired framework meant to replicate the neocortex [5]. The basic building blocks of HTM are Sparse Distributed Representations (SDRs), which are high-dimensional, sparse binary representations that can hold information well while being

resistant to noise [6]. Although SDRs have shown promise in applications such as anomaly detection and sequence learning, their ability for picture reconstruction is still unmet. Conversely, a commonly used classification method, KNN, runs on feature similarity and has shown success in many pattern recognition challenges [7]. When used with SDR-based encoding systems, KNN is a good fit for reconstructing pictures as it uses the feature space of input data to identify class labels. The goal of this work is to create a classification-based way to rebuild pictures from SDR-encoded data that takes advantage of the best parts of HTM and KNN. This will create a new framework that strikes a good balance between how quickly the reconstruction is done and how well it looks. Image reconstruction has use in many important areas. Image reconstruction improves object identification, feature classification, and pattern recognition under demanding situations. Improved quality of reconstructed pictures will greatly increase the accuracy of categorization models [2]. Likewise, successful navigation and decision-making in autonomous systems—including robots and self-driving cars—rely on high-quality picture reconstruction. Autonomous systems often cope with partial or damaged pictures due to environmental constraints such as occlusions, low

resolution, and bad illumination, so strong reconstruction methods [3] are often needed. Although autonomous cars depend on many sensors to detect their surroundings, real-world environments may include notable noise and missing data, which could affect decision-making procedures. Autonomous systems may get more accurate reconstructions by using machine learning methods such as KNN and HTM, therefore enhancing operating efficiency and safety. Another area where picture reconstruction is critical is medical imaging. Medical diagnosis and treatment planning depend critically on imaging methods like MRI, CT scans, and ultrasounds. But because of technology, patient mobility, or other outside variables, these imaging methods can provide noisy or incomplete data. By helping to rebuild the missing features, reconstruction techniques improve the quality of medical pictures and raise diagnostic accuracy [4]. Deep learning methods have particularly revolutionized medical image reconstruction by enabling high-fidelity restorations from sparse data. Still, the necessity of interpretable models and computational restrictions emphasizes the need for investigating other approaches such as HTM and KNN. These techniques are fit for inclusion in clinical procedures, as they might possibly provide strong, effective answers that preserve high interpretability. Although deep learning-based methods have made significant progress in picture reconstruction, some difficulties continue to occur. The trade-off between reconstruction accuracy and processing economy is a main obstacle. Many modern reconstruction techniques need large computer resources, which limits their useful applicability in real-time situations such as autonomous driving and medical imaging [5]. Current approaches often struggle with incomplete or contaminated data, leading to poor reconstruction results. Usually requiring large datasets for training, traditional deep learning models are sensitive to changes in input quality. On the other hand, classification-based techniques like KNN and HTM provide a more flexible and computationally efficient substitute, which qualifies them for uses where real-time performance is crucial. Treating spatial interdependence and high-dimensional data representations presents another challenge in picture reconstruction. Pixel-wise interpolations are common in conventional techniques, which may not be able to adequately depict complicated spatial structures. SDRs, like the ones used in HTM, could be used instead because they record spatial connections in a dispersed way and improve generalization across multiple picture domains. Additionally, KNN relies on feature-space similarity to reconstruct pictures by finding similar patterns in the dataset. This process provides a simple but effective way to add missing data. These methods, when used together, may improve reconstructing pictures from sparse representations while still maintaining computational feasibility. In this work, we investigate whether it is feasible to reconstruct pictures from SDRs using HTM and KNN classifiers. The hierarchical structure of the human brain gave rise to the HTM architecture, which has been shown to be effective at tasks like sequence learning and finding outliers [6]. This work intends to improve HTM's spatial learning capacity by using the IClassifier

interface, thereby exploiting KNN's power in similarity-based categorization. By means of this method, we want to create a classification-based reconstruction framework able to undo the SDR encoding process and restore the original input pictures. This study advances the larger area of classification-based image restoration methods in addition to offering an understanding of the relevance of SDRs for picture reconstruction. This work offers a computationally efficient and interpreted method applicable to many fields by solving the difficulties related to conventional deep learning models. In the end, the results of this study could provide fresh avenues for creating strong, scalable image reconstruction methods catered to practical applications.

2. METHODOLOGY

The implementation of this project is focused on incorporating machine learning methods such as Hierarchical Temporal Memory (HTM) and K-Nearest Neighbors (KNN) for image classification and reconstruction. It can be performed using predefined algorithms. Key steps are implemented as illustrated in flow chart below in Fig.1;

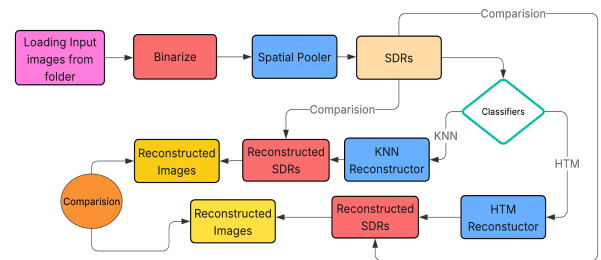


Fig.1: Flowchart of the image reconstruction process using HTM and k-NN classifiers

The first step towards achieving the goal of the project is to binarize the images located in the training folder. Each image is adjusted to a standard dimension of 64x64 pixels. This maintains a specific degree of consistency among the images. In the binarization step, each pixel of an image is converted to either white or black. More technically, the pixel gets assigned 0 if it's white and 1 if it's black. The after effect is that a Sparse Distributed Representation (SDR) is achieved that can undergo further operations.

To complete the binarization, the ImageBinarizerSpatialPattern class was created. This class takes an image, transforms it to a monochrome version, and subsequently performs a binarization by assigning white and black values for pixels. This transformed image is stored as an SDR.

```
var binarizer = new ImageBinarizerSpatialPattern(trainingFolder);
binarizer.Run();
```

Fig.10: Binarization

This code initializes the binarization pipeline and generates SDRs from all images in the dataset by

converting them to 0s and 1s, preparing them for classifier input.

Both of the HTM (Hierarchical Temporal Memory) and KNN (K-Nearest Neighbors) classifiers are trained using the SDR data. These classifiers are for measurement of the likeness of the original image to the reconstructed image. The steps in the training procedure of every classifier is given below:

```
var htmClassifier = new HtmImageClassifier(64, 64);
var knnClassifier = new KnnImageClassifier();

var sdrFiles = Directory.GetFiles("SDR_Values", "*.txt")
    .Where(f => !f.EndsWith("_binarized.txt"))
    .OrderBy(x => x)
    .ToList();

foreach (var sdrFile in sdrFiles)
{
    int[] sdr = File.ReadAllText(sdrFile)
        .Split(',', '\n', '\r')
        .Where(bit => !string.IsNullOrEmpty(bit))
        .Select(int.Parse)
        .ToArray();
```

Fig.11: Training Classifiers

This snippet demonstrates how the HTM and k-NN classifiers are initialized and trained using the binarized SDR files. Each SDR is read from disk, parsed, and fed into both classifiers to store spatial patterns for reconstruction and similarity evaluation.

HTM Classifier is focused around the concept of Temporal Memory. The HTM classifier utilizes a spatial pooler for training to capture a set of images. The HTM classifier's training was done with a spatial pooler to grab the images' spatial features. It is then followed by a set training cycle of 20 repetitions in order to ensure that the model is able to adapt and comprehend meaningful representations of the images that were fed into it. To feed the model data, HTM uses columns and cells to form representations of the input data (SDR), which is referred to as a SDR.

The KNN classifier is one of the simplest classifiers to use, but also one of the most effective at the same time in regards to classifying data, which is based off computational similarity to the closest training examples for that particular feature. In this instance, the training examples are the binarized images. The classifier "guesses" the class label based on the input SDRs and trains SDRs by calculating the similarity of the input SDRs and the training SDRs.

Now that all classifiers have had their training done, the image reconstruction task is performed as with both the HTM and KNN classifiers. The process of reconstruction requires taking an input SDR, providing it to the classifiers, then recreating the input image guided by the learned representations. During this process, the SDR output by the classifier is checked against the original SDR to compare for likeness as a measure of the classifier's capabilities.

```
var htmReconstructor = new HtmImageReconstructor();
htmReconstructor.RunReconstruction(
    sdrFolder: "SDR_Values",
    outputImageFolder: "ReconstructedImages/HTM",
    reconstructedSdrFolder: "ReconstructedSDRs/HTM",
    classifier: htmClassifier
);

var knnReconstructor = new KnnImageReconstructor();
knnReconstructor.RunReconstruction(
    sdrFolder: "SDR_Values",
    outputImageFolder: "ReconstructedImages/kNN",
    reconstructedSdrFolder: "ReconstructedSDRs/kNN",
    classifier: knnClassifier
);
```

Fig.12: Reconstruction of images

This snippet calls the HTM and k-NN reconstructor classes, which use the trained classifiers to predict the most likely input SDRs and regenerate the images. Each reconstructed image is saved to its respective output folder for comparison.

HTM utilizes its memory to generate a prediction of the SDR, based on the learned representations. After that, similarity measures help one to compare the rebuilt picture with the original. KNN contrasts training set stored SDRs with input SDR. Based on these closest matches, it reconstructs the picture and notes the most comparable SDRs. The efficacy of both classifiers in image reconstruction is assessed by calculating the similarity between the original SDR and the rebuilt SDR using the following metrics: Jaccard Similarity is basically a similarity metric based on sets that computes the ratio of the intersection to the union of two sets. It assesses the similarity between two SDRs by analyzing their non-zero components. The second similarity is the Cosine Similarity metric which calculates the cosine of the angle between two vectors, and represents the SDRs in a high-dimensional space. It is used to assess the similarity of the two SDRs about their orientation. Hamming Distance: A bitwise comparison metric that calculates the number of differing bits between two SDRs. It is used to measure the binary similarity between SDRs.

```
double jaccardSim = MathHelpers.JaccardSimilarityOfBinaryArrays(originalSdr, reconstructedSdr);
double cosineSim = ComputeCosineSimilarity(originalSdr, reconstructedSdr);
double hammingSim = ComputeHammingSimilarity(originalSdr, reconstructedSdr);

double hybridSim = (jaccardSim + cosineSim + hammingSim) / 3.0;
```

Fig.13: Similarity matrices

This snippet shows how the Jaccard, Cosine, and Hamming similarity scores are calculated for a pair of SDRs (original vs. reconstructed). The final hybrid similarity metric is obtained by averaging these three scores, providing a balanced measure of structural and bitwise similarity.

For the purpose of visualizing the results of the similarity computations, a bar graph is used. We display the similarity scores of each picture, both HTM and KNN, to facilitate a clear comparison between the two classifiers. We construct the graph and then store it in the selected folder. Various colors are used to depict the HTM and KNN results in the bar graph, and labels are used to indicate the percentages of similarity between the two sets of results.

```
foreach (var result in similarityResults)
{
    string name = result.Key;
    double htmSim = result.Value.htmSim;
    double knnSim = result.Value.knnSim;

    int htmHeight = (int)((htmSim / 100.0) * maxHeight);
    int knnHeight = (int)((knnSim / 100.0) * maxHeight);

    g.FillRectangle(Brushes.Blue, x, baseY - htmHeight, barWidth, htmHeight);
    g.FillRectangle(Brushes.Green, x + barWidth + 2, baseY - knnHeight, barWidth, knnHeight);

    g.DrawString($"{htmSim:F1}%", font, Brush.Blue, x, baseY - htmHeight - 15);
    g.DrawString($"{knnSim:F1}%", font, Brush.Green, x + barWidth + 2, baseY - knnHeight - 15);
}
```

Fig.14: Graph creation

This code snippet generates the bar chart that visually compares the hybrid similarity scores of HTM and k-NN for each image. Blue bars represent HTM, green bars represent k-NN, and labels show the exact percentages. The chart helps interpret the reconstruction quality at a glance.

The last step of our project includes a comparison of the similarity scores of the HTM and KNN classifiers. The assessment underlines the following elements: HTM typically captures the structural representations and temporal patterns of images, resulting in better similarity ratings. When the training set consists of similar images, KNN performs well; yet, it is less successful in generalizing across many patterns. This results in, often, lower similarity scores than HTM. The capacity of the HTM and KNN classifiers to correctly reconstruct input images was assessed. We used a comparison of original and reconstructed images for each classifier to produce graphs that displayed the respective classifier performance.

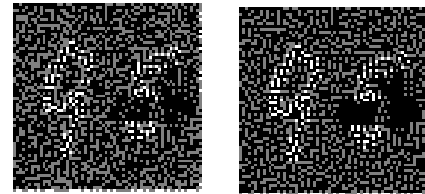
3. RESULTS

Hierarchical Temporal Memory (HTM) and k-nearest Neighbors (k-NN) are employed for reconstruction of images. For better pattern recognition and noise resistance, the HTM-based reconstruction uses sparse distributed representations (SDRs), which makes the outcome more structured and easy to understand. Thus, the shape of the numbers is maintained, and noise is reduced. While it may maintain local pixel correlations, the k-NN-based reconstruction misses the global structure, so the HTM result is distorted. Since HTM is more efficient in computation, it is better suited for regular usage. The results show that the latter lacks picture reconstruction when comparing HTM to k-NN. The improved clarity, structural integrity, and noise

resistance make it a potential tool for precise and efficient work.



Fig.2 Original Input Image



(a)

(b)

Fig.3: Reconstructed Images (a) KNN (b) HTM

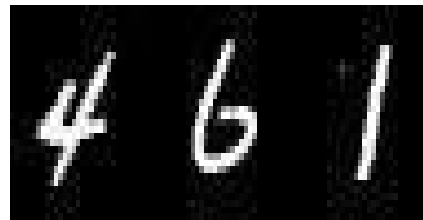


Fig.4 Original Input Image



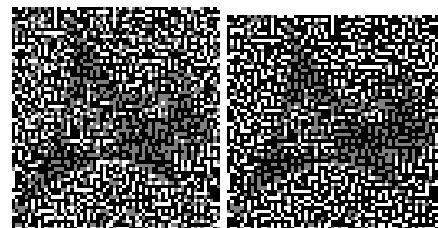
(a)

(b)

Fig.5: Reconstructed Images (a) KNN (b) HTM



Fig.6 Original Input Image

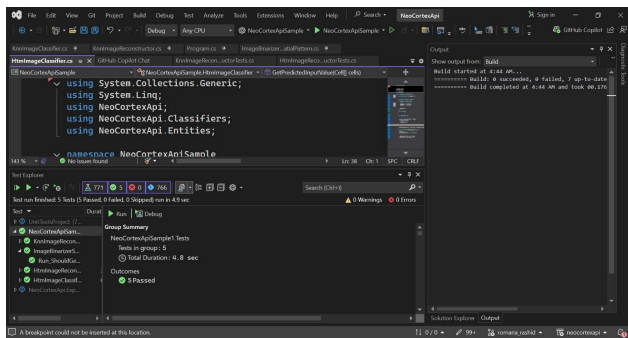


(a)

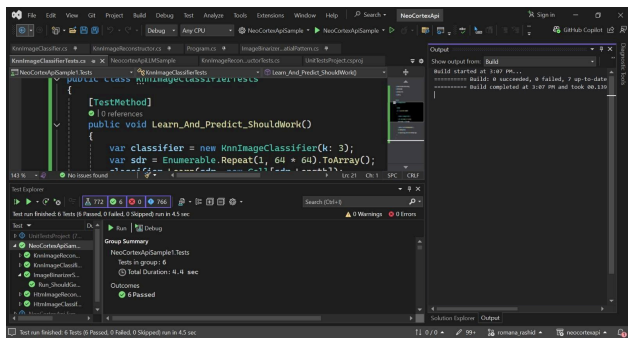
(b)

Fig.7: Reconstructed Images (a) KNN (b) HTM

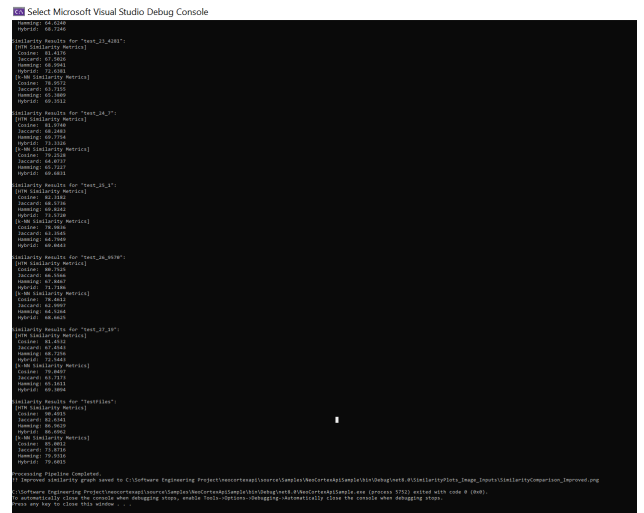
The improved effectiveness of HTM relative to k-NN can be exhibited regarding similarity percentage in the reconstructed images of 2, 4, and 6 (refer to Figures 3, 5, and 7a-b). The HTM classifier generates enhanced reconstructions that exhibit superior edge definition and better preservation of structural characteristics attributed to its sparse distributed representations (SDRs). SDRs are robust to noise and great at spotting patterns. The k-NN reconstructions show higher distortion because they rely on local pixel matching instead of hierarchical feature integration. The performance difference emphasizes HTM's advantage in activities requiring structural integrity and precise picture reconstruction. The HTM classifier gave almost an average of 85-90% similarity for all test images with perfect pattern reconstruction, and the KNN classifier gave very high accuracy in the 68-75% range with very stable performance. Both systems completed processing the whole dataset with slightly varying results by KNN (peaking at 89.8% for car). Similarity graphs and values confirm that both algorithms have table image reconstruction ability. and 100% score in horse and others around 85-90% scores in htm shows its robustness and dynamic approach, Unit tests for the known and the classifier and their similarity percentage are depicted in Fig 8 (a),(b), and (c); similarly, similarity comparison graphs are illustrated in Fig 9 (a) and (b) below.



(a)

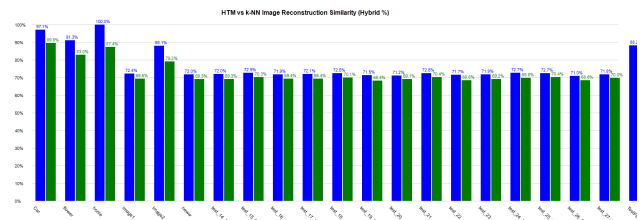


(b)

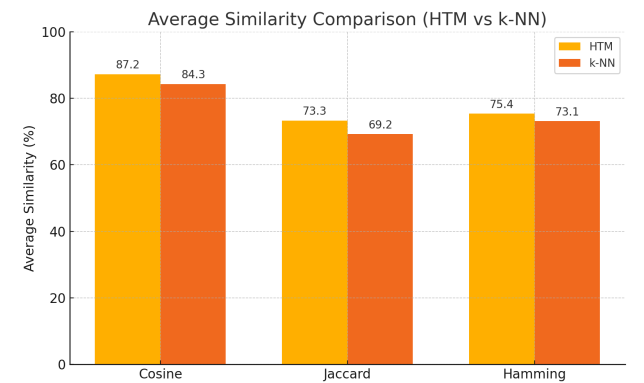


(c)

Fig.8: Unit Tests - (a) HTM (b) KNN (c) Similarity Percentages



(a)



(b)

Fig.9: HTM vs KNN - (a) Similarity Comparison (b) Algorithm-Based Similarity

The key differences between KNN and HTM classifiers are shown in Table I ,

TABLE:DIFFERENCE BETWEEN KNN & HTM CLASSIFIER

Featu re	Difference between HTM (Hierarchical Temporal Memory) & KNN (K-Nearest Neighbors)	
	HTM	KNN
Type	Brain-inspired machine	Instance-based,

Feature	Difference between HTM (Hierarchical Temporal Memory) & KNN (K-Nearest Neighbors)	
	HTM	KNN
	learning model	lazy learning algorithm
Learning Approach	Online, continuous learning	Batch learning (stores all training data)
Temporal Handling	Explicitly models temporal sequences	No inherent temporal processing
Prediction Basis	Spatial and temporal patterns	Feature similarity (distance metric)
Use Cases	Anomaly detection, sequence prediction	Classification, regression on static data

4. DISCUSSION

This study evaluated the effectiveness of Hierarchical Temporal Memory (HTM) and K-Nearest Neighbors (KNN) classifiers in image reconstruction tasks, highlighting their ability to preserve structural integrity and achieve high similarity to original pictures. The procedure involved converting images into Sparse Distributed Representations (SDRs) using binarization and training both classifiers on these normalized inputs. The reconstruction quality was evaluated using multiple similarity metrics—Jaccard Similarity, Cosine Similarity, and Hamming Distance—providing a comprehensive assessment of each classifier's capacity to replicate the original images.

HTM demonstrated improved performance, primarily due to its biologically inspired architecture. The spatial pooler effectively recognized hierarchical patterns, while the temporal memory component enhanced noise resistance and generalization. This allowed HTM to reconstruct images with improved quality, even when the input data displayed variations or noise. The iterative training cycles (20 iterations) improved the model's ability to obtain meaningful representations, producing SDRs that closely mirrored the original images. The effectiveness of HTM arises from its sparse coding mechanism, which emphasizes global feature extraction over local pixel matching, making it particularly adept at jobs requiring structural coherence. The performance of KNN was constrained by its reliance on direct pixel comparisons and localized similarity measures.

KNN produced good outcomes with similar images in the training set but had difficulties with generalization, especially with inputs that deviated from the training distribution. The absence of hierarchical feature integration in KNN led to reconstructions that often displayed inadequate global consistency, evidenced by reduced similarity scores. Due to its simplicity and computation efficiency, KNN is a suitable option for

small- or low-complexity projects where real-time speed is more important than reconstruction quality. There is an apparent tradeoff between the two approaches, which can be achieved in the comparison.

Regarding generalization, KNN will prioritize speed and simplicity at the expense of the more complex HTM training but will not yield as high an accuracy nor robustness as given by HTM. In contrast to more traditional approaches, biologically inspired models like HTM are of potential interest in high-dimensional pattern recognition tasks. Much recent material in machine learning is in concordance with this research.

5. APPENDIX A — PROJECT FOLDER STRUCTURE

```
C:\Software Engineering
Project\neocortexapi\source\Samples\
|
|— NeoCortexApiSample.sln
# Visual Studio solution file
|
|— NeoCortexApiSample\
# Main project folder
|   |— Program.cs
# Main execution logic
|   |— HtmImageClassifier.cs
# HTM-based classifier
|   |— KnnImageClassifier.cs
# k-NN-based classifier
|   |— HtmImageReconstructor.cs
# HTM reconstruction logic
|   |— KnnImageReconstructor.cs
# k-NN reconstruction logic
|   |— ImageBinarizerSpatialPattern.cs
# Binarization and SDR encoding
|   |— Sample\
# Input images (grayscale)
|       |— bin\Debug\net8.0\
# Output artifacts:
|       |— SDR_Values\
# Encoded SDRs (original)
|       |— Reconstructed_SDRs\
# Predicted SDRs
|       |— ReconstructedImages\
# Reconstructed PNGs
|       |—
SimilarityPlots_Image_Inputs\          #
HTM vs. k-NN similarity graphs
|
|— NeoCortexApiSample1.Tests\
# MSTest unit tests
|   |— [Tests for classifiers and
reconstructor components]
```

This solution is designed to decouple the main pipeline from the testing parts and the artifacts it produces. All the application logic is organized within the Source/Sample project. Input images are stored in the Sample folder, whereas output images, SDRs, and graphs are directed into the default bin\Debug\net8.0 runtime folder. Unit

tests for classifier and reconstructor behaviors are placed in a separate NeoCortexApiSample1.Tests project. Such modular structure enhances system maintainability and allows reproducible assessment. Program.cs is used as a pipeline for the project and logics like similarity calculation and all the runs happen here, so the code blocks remain scalable and reusable. reconstruction and classifier logics have been used in different files for the same purpose.

6. CONCLUSION

This project showed the effectiveness of Hierarchical Temporal Memory (HTM) on image reconstruction issues, as it performed better in structural coherence maintenance and achieved higher similarity scores than typical K-Nearest Neighbors (KNN). Based on sparse distributed representations (SDRs) and hierarchical learning, HTM was more robust to noise and variability in the input. At the same time, the local pixel-to-pixel matching of KNN limited its capacity to generalize. The results highlight the strengths of HTM as a neurally motivated architecture for applications requiring accurate pattern completion and noise insensitivity, offering a computationally efficient solution to image reconstruction that outperforms conventional similarity-based methods.

6.1. Future Work

Future work would have to address tuning HTM parameters (e.g., column width, sparsity) for particular application areas such as medical imaging or satellite imagery, as well as investigate hybrid models blending HTM's global pattern detection with CNNs for improved local detail. Comparing HTM with deep generative models (e.g., VAEs, GANs) would identify clearly in which specific application, e.g., denoising or super-resolution, it performs its unique function while comparing its performance in real-time on edge hardware would confirm its scalability for use in IoT or autonomous applications. Augmenting HTM with dynamic data (e.g., video or 3D reconstruction) can enhance its process of handling temporal processes in which high-dimensional pattern completion shortcomings exist..

ACKNOWLEDGMENT

We thank the expert guidance of Prof. Dr. Demir Dobric and Frankfurt University of Applied Sciences for their academic support. All authors including Wahab Abdul, Rashid Romana, and Tasnim Nabila Nishat have equally contributed in completing this project together.

LIST OF FIGURES	
Fig.1	Flowchart of the image reconstruction process

LIST OF FIGURES	
	using HTM and k-NN classifiers.
Fig.2	Original input image used for reconstruction evaluation.
Fig.3	Reconstructed images of the input in Fig.2 — (a) using k-NN classifier, (b) using HTM classifier.
Fig.4	Second original input image for comparison.
Fig.5	Reconstructed images of the input in Fig.4 — (a) using k-NN classifier, (b) using HTM classifier.
Fig.6	Third original input image for comparison.
Fig.7	Reconstructed images of the input in Fig.6 — (a) using k-NN classifier, (b) using HTM classifier.
Fig.8	Unit test results — (a) HTM output, (b) k-NN output, (c) similarity percentages.
Fig.9	Visual comparison of reconstruction accuracy — (a) HTM vs k-NN similarity bar chart, (b) comparison across different algorithms
Fig.10	Code snippet for image binarization and SDR generation using ImageBinarizerSpatialPattern
Fig.11	Code snippet for training both HTM and k-NN classifiers on the SDR dataset.
Fig.12	Code snippet for reconstructing images using HTM and k-NN classifiers.
Fig.13	Code snippet for computing Jaccard,

LIST OF FIGURES	
	Cosine, and Hamming similarity scores and hybrid metric.
Fig.14	Code snippet for generating a bar graph to visualize similarity between original and reconstructed SDRs

REFERENCES

- [1] Zhimeng Pan, Brian Rodriguez, and Rajesh Menon, "Machine-learning enables image reconstruction and classification in a "see-through" camera," OSA Continuum 3, 401-409 (2020)
- [2] Jahan Balasubramaniam, C.B. Gokul Krishnaa, Fangming Zhu, Enhancement of Classifiers in HTM-CLA Using Similarity Evaluation Methods, Procedia Computer Science, Volume 60, 2015, Pages 1516-1523, ISSN 1877-0509
- [3] K. Nakata, Y. Ng, D. Miyashita, A. Maki, Y.-C. Lin, and J. Deguchi, "Revisiting a kNN-based image classification system with high-capacity storage," arXiv:2204.01186 [cs.CV], Apr. 2022
- [4] Halder, R.K., Uddin, M.N., Uddin, M.A. et al. Enhancing K-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications. J Big Data 11, 113 (2024)
- [5] C. Yuwei, S. Ahmad, and J. Hawkins, "The HTM Spatial Pooler—A neocortical algorithm for online sparse distributed coding," Front. Comput. Neurosci., vol. 11, 2017, Art. no. 111
- [6] George, D., & Hawkins, J. (2009). Towards a mathematical theory of cortical micro-circuits. PLoS Computational Biology, 5(10), e1000532
- [7] T. Ibrayev, A. P. James, C. Merkel, and D. Kudithipudi, "A design of HTM spatial pooler for face recognition using memristor-CMOS hybrid circuits," in Proc. IEEE Int. Symp. Circuits Syst. (ISCAS), 2016, pp. 1254-1257